

Fetch API with useEffect (Detailed)

What is Fetch API?

The **Fetch API** is a built-in JavaScript method used to send HTTP requests to a server and retrieve data.

In React, API calls are usually made inside the **useEffect()** hook because we want the data to be loaded when the component is rendered.

Real World Example

Think about an Amazon application.

1. User opens the Products page.
2. React component renders.
3. React sends a request to the backend.
4. Backend fetches data from the database.
5. Backend returns JSON data.
6. React stores the data in state.
7. UI automatically displays all products.

User Opens Products Page



React Component Loads



`useEffect()`



`fetch()`



FastAPI / Node Backend



Database



JSON Response



`setProducts()`



React Re-renders



Display Product Cards

Step 1. Import Hooks

```
import { useEffect, useState } from "react";
```

Explanation

Hook	Purpose
useState()	Stores data inside the component
useEffect()	Performs side effects like API calls

Step 2. Create State

We need three states.

```
const [products, setProducts] = useState([]);  
const [loading, setLoading] = useState(true);  
const [error, setError] = useState("");
```

Explanation

Products State

```
const [products, setProducts] = useState([]);
```

Stores all products received from the API.

Initially

```
[]
```

After API

```
[  
  { id:1, title:"iPhone" },  
  { id:2, title:"Laptop" }  
]
```

Loading State

```
const [loading, setLoading] = useState(true);
```

Shows a loading message while waiting for the server response.

Loading...

Error State

```
const [error, setError] = useState("");
```

Stores an error message if the API request fails.

Example

Failed to fetch products.

Step 3. Fetch Data using useEffect()

```
useEffect(() => {  
  const fetchProducts = async () => {  
    try {  
      setLoading(true);  
  
      const response = await fetch(  
        "https://dummyjson.com/products"  
      );  
  
      const data = await response.json();  
  
      setProducts(data.products);  
    } catch (err) {  
      setError("Failed to fetch products.");  
    } finally {  
      setLoading(false);  
    }  
  };  
  
  fetchProducts();  
}, []);
```

Understanding Every Line

useEffect()

```
useEffect(() => {  
  
}, []);
```

Runs only once after the component is rendered because of the empty dependency array ([]).

Async Function

```
const fetchProducts = async () => {
```

Creates an asynchronous function.

Why?

Because API requests take time and JavaScript should wait for the response.

Try Block

```
try {
```

Contains the code that may throw an error.

Loading State

```
setLoading(true);
```

Before sending the request,

Loading...

is displayed.

Fetch API

```
const response = await fetch(  
  "https://dummyjson.com/products"  
);
```

`fetch()` sends an HTTP GET request.

`await` pauses execution until the server responds.

Convert JSON

```
const data = await response.json();
```

Server response

JSON



JavaScript Object

Save Products

```
setProducts(data.products);
```

Stores products inside React state.

React immediately re-renders the UI.

Catch Block

```
catch(err) {  
    setError("Failed to fetch products.");  
}
```

If

- Internet is disconnected
- Server crashes
- Wrong URL

React displays the error.

Finally Block

```
finally{  
    setLoading(false);  
}
```

Runs whether the API succeeds or fails.

Stops the loading message.

Complete Flow

Component Loads



useEffect()



```
fetchProducts()  
  ↓  
setLoading(true)  
  ↓  
fetch()  
  ↓  
Server  
  ↓  
response.json()  
  ↓  
setProducts()  
  ↓  
React Re-render  
  ↓  
Products Displayed
```

Step 4. Handle Loading

```
if (loading) {  
  return <h2>Loading Products...</h2>;  
}
```

Until data arrives,

Loading Products...

is displayed.

Step 5. Handle Errors

```
if (error) {  
  return <h2>{error}</h2>;  
}
```

Example

Failed to fetch products.

Step 6. Display Products

```
return (
```

```
<div className="container mt-4">

<h2 className="text-center mb-4">
Product Listing
</h2>

<div className="row">

  {products.map((product) => (

    <div
      key={product.id}
      className="col-md-4 mb-4"
    >

      <div className="card h-100 shadow">

        <img
          src={product.thumbnail}
          className="card-img-top"
          height="220"
          alt={product.title}
        />

        <div className="card-body">

          <h5>{product.title}</h5>

          <p>

            Price :
            <strong> ₹ {product.price}</strong>

          </p>

          <p>

            Rating :
            ★ {product.rating}

          </p>

          <button className="btn btn-primary w-100">

            Buy Now

          </button>

        </div>

      </div>

    )
  )}

</div>
```

```
</div>
```

```
)) }
```

```
</div>
```

```
</div>
```

```
);
```

Output

📱 iPhone	🖥 Laptop	🎧 Headphones	
----- ----- -----			
₹ 999	₹ 1299	₹ 199	
⬜ 4.8	⬜ 4.6	⬜ 4.5	
Buy Now	Buy Now	Buy Now	

Concepts Covered

- ☐ `useEffect()`
 - ☐ `useState()`
 - ☐ `fetch()`
 - ☐ `async / await`
 - ☐ `try...catch...finally`
 - ☐ Loading State
 - ☐ Error Handling
 - ☐ `map()`
 - ☐ Bootstrap Grid System
 - ☐ Bootstrap Cards
-

Common Beginner Mistakes

- ☐ Forgetting the empty dependency array (`[]`), causing repeated API calls.

- ☐ Forgetting `await` before `fetch()` or `response.json()`.
 - ☐ Not handling loading and error states.
 - ☐ Using `products.map()` before the data is loaded.
 - ☐ Forgetting to add a unique key inside `map()`.
-

Interview Questions

1. Why do we fetch API data inside `useEffect()` instead of directly inside the component?

Answer:

If we call `fetch()` directly inside the component body, it runs every time the component renders. Since updating state causes another render, this can create repeated API requests or even an infinite loop. By placing the API call inside `useEffect()` with an empty dependency array `[]`, React runs it only once when the component is first mounted.

2. Why do we use `async` and `await` with Fetch API?

Answer:

API requests take time to complete. `async` allows us to write asynchronous code, and `await` pauses execution until the server responds. This makes the code easier to read and avoids deeply nested callbacks.

3. Why do we use `try...catch...finally`?

Answer:

- `try` executes the API request.
- `catch` handles errors such as network failures or invalid URLs.
- `finally` runs regardless of success or failure, making it the ideal place to stop the loading indicator.